

Algoritmi e Strutture Dati–parte I

lez. 2

Francesco Camastra

francesco.camastra@uniparthenope.it

Docente

- Prof. Francesco Camastra

Orario lezioni

- Mercoledì 11.00-13.00
- Venerdì 9.00-11.00

Ricevimento

- Orario: Giovedì 10.00-14.00
- Dove: Stanza 430, Quarto piano, Centro Direzionale, Isola C4 o su Teams
- Per il ricevimento è **OBBLIGATORIO** inviare una email di prenotazione almeno due giorni prima del giorno di ricevimento.
- email del Docente:
francesco.camastra@uniparthenope.it
- Pagina del Docente:
dist.uniparthenope.it/francesco.camastra

Prerequisiti

- Programmazione I
- Matematica I
- Programmazione II
- **NOTA BENE:** Le propedeuticità non sono obbligatorie ... Ossia potreste anche sostenere l'esame senza aver dato Programmazione I e Programmazione II

Frequenza

- La Frequenza del corso non è obbligatoria

Modalità dell' esame

- Prova scritta di Algoritmi e Strutture Dati I ovvero Due prove intercorso (validità quattro appelli)
- Svolgimento di un progetto Algoritmi e Strutture Dati II da scegliersi tra una lista di progetti che verrà comunicata dal docente (validità quattro appelli).
- Superato lo scritto di ASD-I ed il progetto di ASD-II si accede all' Esame Orale congiunto di ASD-I e ASD-II

Testi Consigliati

- ALGORITMI:
- Cormen, Leiserson, Rivest, Stein
*Introduzione agli Algoritmi e
alle strutture Dati*
(seconda edizione o terza edizione)
Ed. Mc Graw-Hill.

Sommario della Lezione:

NOTAZIONI ASINTOTICHE

- Definiremo le notazione asintotiche O , Ω , Θ . Tali notazioni ci permetteranno di valutare la crescita di funzioni elementari
- Analizzeremo il comportamento asintotico delle più comuni funzioni utilizzando la notazione O .

Una Prima Domanda

- Perché un informatico deve conoscere le notazioni asintotiche ?

Risposta

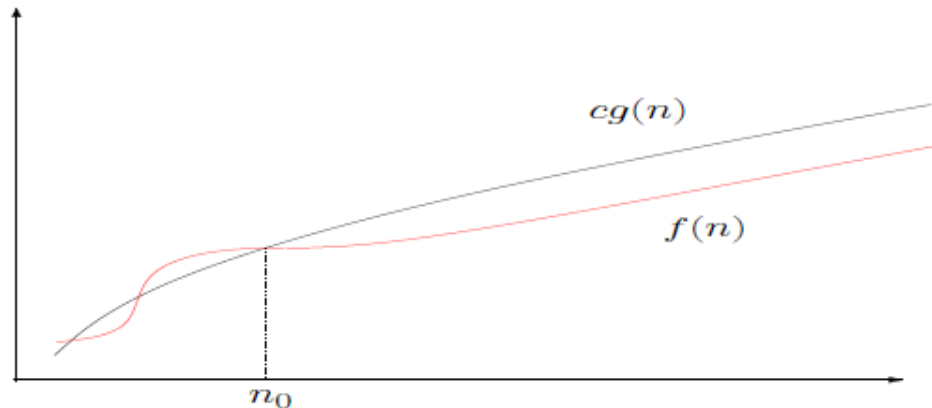
- L' informatico deve stimare la velocità di crescita di funzioni che rappresentano l' ammontare delle risorse (computazionali, o di memoria) richieste dagli algoritmi che ha progettato.
- Pertanto l' informatico utilizzando le notazioni asintotiche può confrontare algoritmi differenti che risolvono lo stesso problema e selezionare, quindi, il più efficiente.

Il comportamento asintotico: Necessità

- Il comportamento asintotico é necessario perché si utilizzano, spesso, input di dimensioni enormi.
- Esempio: L'ordinamento di un vettore numerico.
 - Se il vettore contiene cento elementi, qualunque algoritmo di ordinamento va bene, in quanto non noteremo differenze apprezzabili tra i tempi di CPU impiegati dai programmi che implementano i vari algoritmi. Se il vettore contiene un milione di elementi, i tempi di CPU impiegati dai programmi sono considerevoli. Pertanto é **necessario selezionare l'algoritmo che richiede meno tempo computazionale.**

La notazione O : Definizione

- Date $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$ diremo che $f(n) = O(g(n))$ se e solo se $\exists c > 0, \exists n_0$ tale che, $f(n) \leq cg(n), \forall n \geq n_0$



Un esempio

- Consideriamo la funzione $f(n) = 5n^3 + n^2 + 3$ verifichiamo che vale $f(n) = O(n^3)$.
- Per verificare l'uguaglianza bisogna dimostrare che $\exists n_0, c$ tale che $f(n) \leq cn^3$. Abbiamo:
$$f(n) = 5n^3 + n^2 + 3 \leq 5n^3 + n^3 + 3 \quad \forall n \geq 1$$
$$f(n) \leq 6n^3 + 3 \leq 6n^3 + n^3 \leq 7n^3 \quad \forall n \geq 2$$

Un Polinomio è O del suo termine di grado più elevato

- Dato un polinomio $P_k(n)$ di grado k allora è $P_k(n) = O(n^k)$.

Dimostrazione:

$$\begin{aligned} a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0 &\leq \\ |a_k| n^k + |a_{k-1}| n^{k-1} + \dots + |a_1| n + |a_0| &\leq \\ |a_k| n^k + |a_{k-1}| n^k + \dots + |a_1| n^k + |a_0| n^k &\leq \\ (|a_k| + |a_{k-1}| + \dots + |a_1| + |a_0|) n^k &\leq c n^k \end{aligned}$$

Un Polinomio è O del suo termine di grado più elevato (cont.)

- Dimostrazione (con le sommatorie):

$$\begin{aligned}\sum_{i=0}^k a_i n^i &\leq \sum_{i=0}^k |a_i| n^i \leq \sum_{i=1}^k |a_i| n^k + |a_0| \\ &\leq \sum_{i=0}^k |a_i| n^k \leq \left(\sum_{i=0}^k |a_i| \right) n^k \leq c n^k\end{aligned}$$

Esempi

- Quindi:

- $n^3 + 1000n^2 + 2000 = O(n^3)$

- $200n^3 + n^5 + 1000n^4 = O(n^5)$

- $100n^2 + n^{\frac{5}{2}} + 70n^{\frac{7}{3}} = O(n^{\frac{5}{2}})$

Ordine di grandezza di funzioni elementari: $\log_2 n = O(n)$

- Occorre provare che $c > 0, \exists n_0$ tale che, $f(n) \leq cg(n), \forall n \geq n_0$
- Si dimostra per induzione:
 - Per $n = 1$ abbiamo $\log_2 1 = 0 \leq 1$
 - Supponiamo che sia vera $\log_2 n \leq n \forall n > 1$ e dimostriamo che $\log_2 n + 1 \leq n + 1 \quad \forall n > 1$
 - Si ha $\log_2(n + 1) \leq \log_2(n + n) \leq \log_2 2n$
 $\leq \log_2 2 + \log_2 n \leq 1 + \log_2 n \leq n + 1$

(cont.)

- E se il logaritmo non è in base 2 ?
 - Si ha $\log_a n = (\log_a 2)(\log_2 n) \leq (\log_a 2)n \Rightarrow$
 $\log_a n = O(n)$
- E se abbiamo $\log_a n^b$ con $b > 1$?
 - Si ha $\log_a n^b = b \log_a n \leq acn \Rightarrow$
 $\log_a n^b = O(n)$

Logaritmi contro Radici

- Si ha: $\forall a > 0, b > 0, c > 0, k > 0 \ (\log_c n^b)^a = O(n^k)$
- Esempi
 - $(\log_2 n^6)^{100000} = O(n^{\frac{1}{10000}})$
 - $(\log_2 n^{6000})^{9000} = O(n^{\frac{1}{10000}})$

Polinomi contro Esponenziali

- Vale $\forall k > 0, a > 1 \quad n^k = O(a^n)$
- Dimostrazione:
 - Osserviamo che $n^k \leq n^{\frac{n}{\log_a n}}, \forall n$ tale che $k \leq \frac{n}{\log_a n}$
 - Quindi $n^k \leq n^{\frac{n}{\log_a n}} \leq (a^{\log_a n})^{\frac{n}{\log_a n}} \leq a^{\frac{n \log_a n}{\log_a n}} \leq a^n$
 $\forall n$ tale che $k \leq \frac{n}{\log_a n}$

Ordinamento tra funzioni

Espressione O	Nome
$O(1)$	costante
$O(\log_a \log_b n)$	log log
$O(\log_a n)$	logaritmica
$O(\sqrt[c]{n}), c > 1$	sublineare
$O(n)$	lineare
$O(n \log_a n)$	$n \log n$
$O(n^k) (k > 1)$	polinomiale
$O(a^n) (a > 1)$	esponenziale
$O(n!)$	fattoriale
$O(n^n)$	n^n

Esempi

- $(\log n)^2 = O(n)$
- $3 \log n^2 = O(\sqrt{n})$
- $\sqrt{n} = O(n)$
- $n^9 = O(3^n)$
- $6^n = O(n!)$

Esponenziali e Fattoriali

- $a^n = O(n^n)$
 - $a^n \leq n^n \quad \forall a \leq n$
- $n! = O(n^n)$
 - $n! \leq n^n$
- $a^n = O(n!)$
 - $n! \cong \left(\frac{n}{e}\right)^n \sqrt{2\pi n} \left(1 + \Theta\left(\frac{1}{n}\right)\right)$ **Formula di Sterling**

Proprietá della notazione O

- $a(n) = O(c(n)) \Rightarrow \lambda a(n) = O(c(n)) \quad \lambda > 0;$
- $a(n) = O(c(n)), b(n) = O(d(n)) \Rightarrow$
 $a(n) + b(n) = O(c(n) + d(n));$
- $a(n) = O(c(n)), b(n) = O(d(n)) \Rightarrow$
 $a(n)b(n) = O(c(n)d(n));$
- $a(n) = O(c(n)), c(n) = O(d(n)) \Rightarrow$
 $a(n) = O(d(n));$